Viva Preparation (25 questions, answers match this code)

1. What problem does MPFT solve? A single TCP connection uses one route and the transfer dies or restarts if that route breaks or the file is damaged. MPFT sends chunks over several TCP connections in parallel, verifies each chunk with SHA-256, survives a dead path and can resume.

2. Why TCP for each path and not UDP? TCP already gives reliable ordered bytes and congestion control and passes through NAT. MPFT only adds chunking, verification and recovery at chunk level. UDP would need our own loss detection and congestion control.

3. What is in the 20-byte header? Magic `MP` (2), version (1), type (1), session id (4), chunk index (4), payload length (4), CRC-32 of the first 16 bytes (4). Defined with `struct.Struct(">2sBBIII")` plus a 4-byte CRC in `protocol.py`.

4. Why a CRC on the header if TCP has a checksum? The CRC protects the framing: a wrong `payload_len` would desynchronise the stream. It also protects against corruption injected above TCP, as our emulator does. A bad header closes the connection.

5. Why SHA-256 per chunk and for the whole file? Per chunk: identifies which chunk to resend (NACK) and lets resume trust bytes already on disk. Whole file: final proof before the `.part` file is renamed.

6. How does work stealing work here? All paths pull indexes from one shared `queue.PriorityQueue`. A path takes a new index only after its previous chunk was ACKed, so a faster path takes more. Measured: with 80/20/5 Mbit/s paths the byte shares were 74/20/6 %.

7. Why work stealing and not round robin? Round robin gives each path equal work, so the slowest path finishes last and a dead path blocks its share. Work stealing adapts without measuring bandwidth.

8. What happens when a path dies mid-transfer? The worker's send or receive raises an error (reset, timeout, bad frame). `fail_path` records the failure time and `requeue` puts the in-flight chunk back with priority 0. The worker closes the socket, backs off and reconnects; after 6 consecutive failures, or if connect fails, the path stays down and the others finish.

9. What are the retry rules? A chunk may be retried 5 times (`MAX_RETRIES`); the 6th failure fails the transfer. Delay before re-queue is `min(0.05*2^(n-1), 1.0)` s. A path reconnects with `min(0.1*2^fails, 2)` s backoff. Per-chunk timeout is 5 s.

10. Why do retried chunks go to the front of the queue? With a normal FIFO queue the retried chunk waited behind all fresh chunks; recovery time was 6.9 s instead of 0.2 s in the failure experiment. The `PriorityQueue` with `(0, idx)` for retries fixed it.

11. How does resume work? On MANIFEST the receiver opens `name.part`, and if it exists with the right size it re-hashes each chunk on disk against the manifest. The MANIFEST_ACK lists the chunks that match, and the sender queues only the rest. A damaged chunk simply fails the hash and is resent.

12. Why not keep a resume state file? Extra state can disagree with the file after a crash. Re-hashing costs one read of the partial file but is always correct.

13. How is the file written without loading it into RAM? Sender: `os.pread` of one chunk when a path takes it, and the manifest is built in one streaming pass. Receiver: `os.pwrite` at `index*chunk_size`, which is thread safe and allows out-of-order chunks.

14. What does the receiver do with duplicate or out-of-order chunks? Out of order: written at its own offset. Duplicate: already in the `held` set, so it is ACKed again and not rewritten (counted in `duplicates`). This makes a lost ACK harmless.

15. When is the file considered complete? When `held` has all chunks the receiver hashes the whole `.part` file and, if it equals `file_sha256`, renames it. The sender then sends DONE after all ACKs and the receiver replies DONE (or ERROR). The rename happens only after verification.

16. Why is DONE a handshake in both directions? The sender cannot know the file was verified just from chunk ACKs. Sending DONE after all ACKs gives a deterministic point: the receiver confirms verification with DONE or reports ERROR.

17. How is the first MANIFEST sent if path 0 is down? `ensure_manifest` uses a lock: the first path that connects sends the MANIFEST and the rest wait. If it fails, the lock is released and another path does it.

18. How is recovery time measured? From the first failure detected (`t_fail`) until every chunk that was in flight on failed paths has been ACKed. In the experiment: 0.15 s. With a silent black hole it would include the chunk timeout.

19. Why does one 40 Mbit/s path only reach about 23 Mbit/s? One chunk in flight per path (stop-and-wait): each 256 KiB chunk costs transmit time plus a round trip before the next. Several paths overlap these waits; larger chunks reduce the relative cost (1 MiB gave 107 Mbit/s on 4 paths vs 90 at 256 KiB).

20. How does the emulator add delay without reordering bytes? Each direction has a reader thread that stamps blocks with a due time (`max(previous_due, now+delay+jitter)`) and a writer thread that sends at that time. Due times never decrease, so order is kept. Rate limit: the writer sleeps `len*8/rate` after each block.

21. What does the emulator NOT do? It does not drop IP packets (TCP would hide that anyway) and corruption is injected above TCP. It only approximates real networks; everything runs on one machine.

22. How do you guarantee corrupted data is never accepted? Every CHUNK is hashed on arrival and compared to the manifest before `pwrite`; a mismatch gives NACK and nothing is stored. The whole-file hash is a second barrier. Tested with `corrupt_rate` 1 %, 5 %, 10 % and by flipping a bit directly.

23. What security does MPFT have? None by default: no encryption or authentication. The manifest file name is reduced to its base name to prevent path traversal, and header length is capped at 64 MiB. Use Tailscale or a VPN for confidentiality.

24. How does the dashboard get its data? Sender and receiver write `status/<role>-<session>.json` atomically (temp file plus `os.replace`) every second. FastAPI's `/api/status` reads the files and marks those not updated for 5 s as stale; the JS page polls every second.

25. What would you improve? A sliding window per path (several chunks in flight), TLS or a keyed MAC, adaptive chunk size, true packet-loss emulation, and resume metadata that survives a sender restart without re-hashing the receiver's file.

Web layer

26. Why two servers per instance? The UI server can start transfers, read local files' uploads and kill emulator paths, so it listens on 127.0.0.1 and checks Host and Origin. The peer server must be reachable by other machines, so it is a separate app on 0.0.0.0 that exposes only `GET /health` and `POST /offer`.

27. How does a Receiver ID work without a server? It is base32 of `host:peer_port`, so the sender decodes it locally. The host is the Tailscale 100.x address if present. A code needs the rendezvous service because six random characters cannot encode an address.

28. What makes a code safe to use? It is generated with `secrets`, has no confusable letters, expires in 10 minutes, is deleted on first use, and failed attempts are limited to five per minute per client. It does not provide encryption.

29. Why does an offer need Accept? Anyone who can reach the peer port could otherwise push files to the receiver. A valid code proves the sender was given the code by the receiver, so that offer is trusted; an ID alone is not a secret.

30. How do the live panels get real data? `mpft/events.py` is a thread-safe bus that sender, receiver and emulator emit into. The UI server subscribes, stores events in a ring buffer and streams them over Server-Sent Events; the page reduces them into state (`MP.reduce`) and redraws. A reload replays the buffer.

31. How does Kill path work, and is it a real failure? The emulator proxies are real TCP listeners in front of the receiver. Killing one closes its sockets, so the sender sees a reset or timeout exactly as in the earlier experiments, requeues the chunk and the other paths finish it. The failure is real TCP, only the network is emulated.

32. How do you avoid overwriting or traversal on the receiver? Offer names are reduced to a safe base name; data arrives in a staging directory and is renamed to a unique name (`name (1).ext`) under a lock. Downloads are served only for names the node recorded, and the resolved path must be inside the output directory.

33. What does the web layer not protect against? There is no encryption or authentication on the peer API or rendezvous service; anyone on the network who knows the receiver address can send an offer (it waits for Accept) and a rogue rendezvous could map a code to the wrong ID. Tailscale limits who is on the network and encrypts it.

34. Why does the sender relay its events instead of the receiver pulling them? The sender can reach the receiver (it just made the offer) but the receiver may not reach the sender behind NAT. The

sender POSTs batches to `/events/{id}` and the reply carries the receiver's new events, so one connection direction gives both people the same log.

35. How do you merge two machines' clocks? During the offer round trip the sender takes t0 and t3, the receiver reports its arrival t1 and reply t2; offset = ((t1-t0)+(t2-t3))/2. The receiver's wait for Accept cancels out. Events keep a per-side sequence number, so order is by corrected time, then side, then sequence; clock error can blur ordering across sides by roughly the network delay but never within a side.

36. What is the session token for? It is returned only in the accept reply and required on `/events`, so a stranger who finds the peer port cannot inject events into someone's dashboard. It is compared in constant time and events are forced to the side the token holder may speak for.

37. How does one Receiver ID work over two networks? It encodes all of the receiver's addresses (Tailscale and LAN). The sender probes each, uses the reachable ones, and assigns paths round-robin, so a transfer can use both networks at once. Advanced lets you force one.

38. What happens if the receiver disappears mid-transfer? Paths fail, chunks are requeued up to the retry limit, the job fails with a message to send again. The receiver keeps the `.part` file in a staging folder named by the file hash, and the next offer of the same file resumes from the chunks whose hashes match.

39. How is the incident table built? A reducer watches the merged events: `path_down` opens an incident, `requeue` events attach chunks to it, `backoff` adds retry delays, and when every attached chunk is acknowledged on another path the incident is recovered; recovery time and the paths that absorbed the work come from those events.

40. Why are path ids not merged between sender and receiver? The receiver numbers every accepted connection, so a reconnect gets a new id, while the sender keeps its own. The dashboard therefore uses only the sender's path events for the Paths table and counts a chunk as done when the sender sees the ACK.

41. How does the chunk map scale to 20,000 chunks? It is a canvas, not DOM elements; above 6000 chunks several chunks share one cell and the cell shows the group's state. Hover and click report exact values for single chunks.

42. How does Replay work? Both sides write every tagged event to `sessions/<id>.jsonl`. Replay loads that file and feeds events to the same reducer used live, up to a virtual clock; scrubbing backwards resets and re-reduces. There is no separate replay code path to drift from the live view.

43. What would you do for real security? Encrypt and authenticate the data plane (TLS or Noise) and the peer API, bind the session token to a key exchange, and replace code lookup by a verified rendezvous. Today Tailscale supplies encryption and device authentication.